TRAVAUX PRATIQUES PROXMOX

Virtualisation & Infrastructure Cloud

Dimitri ANGELY

M2 Infrastructure Cloud — Juin 2026

Sommaire

TP1 — Installation et configuration de Proxmox VE
- Partie 1 — Préparation de l'hôte Proxmox
- Partie 2 — Création d'une VM Linux propre
- Partie 3 — Installation de QEMU Guest Agent
- Partie 4 — Nettoyage et préparation du template
- Partie 5 — Conversion en template
- Partie 6 — Création de clones
- Partie 7 — Personnalisation des VMs
- Partie 8 — Snapshot, panne volontaire et rollback

TP2 — Architecture réseau segmentée avec VLANs
- Partie 1 — Création des réseaux Proxmox
- Partie 2 — Déploiement des VMs
- Partie 3 — Configuration du firewall fw01
- Partie 4 — Configuration des VMs par zone
- Partie 5 — Tests de connectivité et filtrage

TP1 — Installation et configuration de Proxmox VE

Partie 1 — Préparation de l'hôte Proxmox

Environnement

Proxmox VE est installé dans une VM VMware Workstation sur un hôte Windows (Intel i7-13650HX, 14 cores / 20 threads, 32 Go RAM). La VM Proxmox est configurée avec 16 vCPUs (1 socket × 16 cores) et 32 Go de RAM.

Configuration réseau de l'hôte

Élément	Valeur
Adresse IP	192.168.131.135/24
Passerelle	192.168.131.2
DNS	192.168.131.2
Interface	vmbr0 (bridge sur ens33)
Mode réseau VMware	NAT

Vérifications post-installation

```
hostnamectl
ip a
ip route
cat /etc/resolv.conf
pvesm status
systemctl status pveproxy
```

Configuration des dépôts

Par défaut, Proxmox 9.x pointe vers des dépôts Enterprise nécessitant un abonnement. Pour un usage lab, on bascule vers les dépôts no-subscription :

```
echo '' > /etc/apt/sources.list.d/pve-enterprise.sources
echo '' > /etc/apt/sources.list.d/ceph.sources
cat > /etc/apt/sources.list.d/pve-no-subscription.sources << 'EOF'
Types: deb
URIs: http://download.proxmox.com/debian/pve
Suites: trixie
Components: pve-no-subscription
EOF
apt update && apt upgrade -y
```

Téléchargement de l'ISO Debian

```
cd /var/lib/vz/template/iso/
wget https://cdimage.debian.org/debian-cd/current/amd64/iso-cd/debian-13.5.0-amd64-netinst.iso
```

Résultat attendu

Interface web accessible sur https://192.168.131.135:8006, dépôts fonctionnels (apt update sans erreur 401), ISO Debian 13.5.0 présente dans local → Images ISO.

Partie 2 — Création d'une VM Linux propre

Configuration de la VM

Paramètre	Valeur
VM ID	9000
Nom	tmpl-debian-13-base
CPU	1 socket × 2 cores (x86-64-v2-AES)
RAM	2048 Mo
Disque	8 Go sur local-lvm (SCSI VirtIO)
Réseau	VirtIO sur vmbr0
Machine	q35
QEMU Agent	Activé

Installation de Debian 13

L'installation se fait depuis l'ISO debian-13.5.0-amd64-netinst.iso. Choix effectués :

- Langue : Français
- Domaine : lab.local
- Mot de passe root : vide (désactivé)
- Utilisateur : labuser
- Partitionnement : disque entier, tout dans une partition, ext4
- Logiciels : serveur SSH + utilitaires système uniquement (pas d'interface graphique)
- GRUB : installé sur /dev/sda

Partie 3 — Installation de QEMU Guest Agent

QEMU Guest Agent permet à Proxmox de récupérer l'adresse IP de la VM, d'effectuer des snapshots cohérents et d'interagir avec l'hyperviseur.

```
sudo -i
apt update && apt upgrade -y
apt install -y qemu-guest-agent cloud-init
systemctl enable qemu-guest-agent
systemctl start qemu-guest-agent
```

> **Vérification**
> Dans Proxmox, le résumé de la VM affiche l'adresse IP — cela confirme que QEMU Guest Agent communique correctement avec l'hyperviseur.

Partie 4 — Nettoyage et préparation du template

Avant la conversion en template, la VM doit être nettoyée pour éviter tout conflit lors du clonage :

```
apt autoremove -y && apt clean
cat /dev/null > ~/.bash_history
truncate -s 0 /etc/machine-id
rm -f /var/lib/dbus/machine-id
ln -s /etc/machine-id /var/lib/dbus/machine-id
rm -f /etc/ssh/ssh_host_*
shutdown -h now
```

Explication de chaque étape :

- machine-id vidé : évite que tous les clones aient le même identifiant réseau
- clés SSH supprimées : chaque clone régénère ses propres clés au premier boot
- historique bash vidé : ne pas transmettre d'informations sensibles aux clones

Ajout du drive Cloud-Init

Depuis l'interface Proxmox : Matériel → Ajouter → Lecteur CloudInit (sur local-lvm)

Configuration Cloud-Init :

- Utilisateur : labuser
- Mot de passe : ChangeMe123
- Configuration IP (net0) : ip=dhcp
- Mettre à jour les paquets : Oui

Partie 5 — Conversion en template

Une fois la VM éteinte et le drive cloud-init ajouté, clic droit sur la VM 9000 → Convertir en modèle.

Le template apparaît avec une icône distincte dans le panneau gauche. Les boutons Démarrer disparaissent — le template est en lecture seule.

> **Pourquoi un template ?**
> Le template garantit que chaque déploiement part d'une base identique, propre et maîtrisée. Cloud-init injecte au premier boot le hostname, l'IP et les clés SSH — chaque clone est donc unique dès son démarrage.

Partie 6 — Création de clones

Différence clone complet vs clone lié

Type	Stockage	Dépendance	Vitesse	Usage
Clone complet	Copie intégrale du disque	Aucune — indépendant	Plus lent	Production, VMs durables
Clone lié	Delta uniquement	Dépend du template	Quasi-instantané	Lab, tests éphémères

VMs créées

VM ID	Nom	Type de clone	Rôle
101	web01	Clone complet	Serveur web
102	db01	Clone lié	Serveur base de données
103	admin01	Clone lié	Machine d'administration
104	test-rollback01	Clone lié	VM de test snapshot/rollback

Partie 7 — Personnalisation des VMs

Adresses IP (attribuées par DHCP via cloud-init)

VM	IP	Service installé
web01	192.168.131.138	Apache2
db01	192.168.131.139	MariaDB 11.8.6
admin01	192.168.131.140	curl, nmap
test-rollback01	192.168.131.141	Apache2

Installation Apache sur web01

```
apt install -y apache2
echo '<!DOCTYPE html><meta charset="UTF-8"><h1>Serveur web01 operationnel</h1>'
> /var/www/html/index.html
```

Test depuis Windows :

```
curl http://192.168.131.138
```

Installation MariaDB sur db01

```
apt install -y mariadb-server
systemctl status mariadb
```

Tests réseau depuis admin01

```
curl http://192.168.131.138          # Test HTTP vers web01
ping -c 3 192.168.131.139            # Test ping vers db01
nmap -p 3306 192.168.131.139         # Test port MariaDB
```

Partie 8 — Snapshot, panne volontaire et rollback

Étapes réalisées sur test-rollback01

1. Installation d'Apache et création d'une page web de test
2. Création du snapshot 'avant-panne' depuis Proxmox (Instantanés → Créer un instantané)
3. Provocation de la panne : arrêt Apache + suppression de la page web
4. Constat de la panne : curl retourne 'Failed to connect to port 80'
5. Rollback : sélection du snapshot 'avant-panne' → Retour en arrière
6. Vérification : curl retourne la page web — VM restaurée

Commandes de la panne

```
systemctl stop apache2
systemctl disable apache2
rm /var/www/html/index.html
```

Vérification après rollback

```
curl http://192.168.131.141
```

Résultat : <!DOCTYPE html><meta charset="UTF-8"><h1>test-rollback01 operationnel</h1>

Conclusion TP1

L'environnement Proxmox est pleinement fonctionnel avec un template Debian 13 réutilisable, 4 VMs clonées et personnalisées, et un cycle snapshot/rollback validé.

TP2 — Architecture réseau segmentée avec VLANs

Partie 1 — Création des réseaux Proxmox

Architecture VLAN

VLAN	Zone	Réseau	Gateway (fw01)
10	MGMT	192.168.10.0/24	192.168.10.254
20	SRV	192.168.20.0/24	192.168.20.254
30	USERS	192.168.30.0/24	192.168.30.254
40	DMZ	192.168.40.0/24	192.168.40.254

Création du bridge vmbr1 VLAN-aware

Depuis Proxmox : proxmox → Réseau → Créer → Bridge Linux

- Nom : vmbr1
- Gère les VLAN : coché (VLAN-aware)
- Démarrage automatique : coché
- Bridge ports : vide (bridge interne uniquement)

Appliquer la configuration pour activer le bridge.

Partie 2 — Déploiement des VMs

VMs créées

VM ID	Nom	Zone	VLAN Tag	IP VLAN
200	fw01	Toutes	10/20/30/40	Voir ci-dessous
201	web-dmz01	DMZ	40	192.168.40.10
202	db-srv01	SRV	20	192.168.20.10
203	client01	USERS	30	192.168.30.10
204	admin-mgmt01	MGMT	10	192.168.10.10

Configuration des interfaces réseau des VMs

Chaque VM (sauf fw01) possède deux interfaces :

- eth0 : connectée à vmbr0 (réseau VMware NAT) — accès SSH de management
- ens19 : connectée à vmbr1 avec le tag VLAN de sa zone — IP statique configurée manuellement

Partie 3 — Configuration du firewall fw01

Interfaces réseau de fw01

Interface Linux	Bridge Proxmox	VLAN	Zone	IP
eth0	vmbr0	-	Management Proxmox	192.168.131.142 (DHCP)
ens19	vmbr1	20	SRV	192.168.20.254
ens20	vmbr1	30	USERS	192.168.30.254
ens21	vmbr1	40	DMZ	192.168.40.254
ens22	vmbr1	10	MGMT	192.168.10.254

Configuration réseau (/etc/network/interfaces)

```
auto ens19
iface ens19 inet static
    address 192.168.20.254
    netmask 255.255.255.0

auto ens20
iface ens20 inet static
    address 192.168.30.254
    netmask 255.255.255.0

auto ens21
iface ens21 inet static
    address 192.168.40.254
    netmask 255.255.255.0

auto ens22
iface ens22 inet static
    address 192.168.10.254
    netmask 255.255.255.0
```

Activation du routage IP

```
echo 1 > /proc/sys/net/ipv4/ip_forward
echo 'net.ipv4.ip_forward=1' >> /etc/sysctl.conf
```

Configuration nftables (NAT + Filtrage)

Le fichier /etc/nftables.conf configure le NAT et les règles de filtrage inter-zones :

```
table inet filter {
    chain forward {
        type filter hook forward priority 0; policy accept;
        # Bloquer USERS vers SRV
        iifname "ens20" oifname "ens19" drop
        # Bloquer SRV vers USERS
        iifname "ens19" oifname "ens20" drop
    }
}
```

```
table ip nat {
    chain postrouting {
        type nat hook postrouting priority 100;
        oifname "eth0" masquerade;
    }
}
```

Explication des règles :

- masquerade sur eth0 : NAT pour l'accès Internet des VMs internes
- drop USERS → SRV : les clients ne peuvent pas accéder directement à la base de données
- drop SRV → USERS : isolation bidirectionnelle entre les deux zones

Partie 4 — Configuration des VMs par zone

web-dmz01 (Zone DMZ — 192.168.40.10)

```
auto ens19
iface ens19 inet static
    address 192.168.40.10
    netmask 255.255.255.0
    gateway 192.168.40.254
apt install -y apache2
echo '<!DOCTYPE html><meta charset="UTF-8"><h1>Serveur web-dmz01 - Zone
DMZ</h1>' > /var/www/html/index.html
```

db-srv01 (Zone SRV — 192.168.20.10)

```
auto ens19
iface ens19 inet static
    address 192.168.20.10
    netmask 255.255.255.0
    gateway 192.168.20.254
apt install -y mariadb-server
```

client01 (Zone USERS — 192.168.30.10)

```
auto ens19
iface ens19 inet static
    address 192.168.30.10
    netmask 255.255.255.0
    gateway 192.168.30.254
```

admin-mgmt01 (Zone MGMT — 192.168.10.10)

```
auto ens19
iface ens19 inet static
    address 192.168.10.10
    netmask 255.255.255.0
    gateway 192.168.10.254
```

Partie 5 — Tests de connectivité et filtrage

Tests de routage inter-VLAN

Source	Destination	Protocole	Résultat	Explication
admin-mgmt01 (MGMT)	192.168.10.254 (fw01)	ICMP	OK - TTL 64	Même zone, pas de routage
admin-mgmt01 (MGMT)	192.168.40.10 (DMZ)	ICMP	OK - TTL 63	Routé via fw01 (1 saut)
admin-mgmt01 (MGMT)	192.168.40.10 (DMZ)	HTTP	OK - Page web	Apache accessible
client01 (USERS)	192.168.30.254 (fw01)	ICMP	OK - TTL 64	Même zone, pas de routage
client01 (USERS)	192.168.40.10 (DMZ)	ICMP	OK - TTL 63	Routé via fw01
client01 (USERS)	192.168.20.10 (SRV)	ICMP	BLOQUE - 100% perte	Regle nftables drop

Explication du TTL 63

Quand client01 ping web-dmz01, le TTL de la réponse est 63 (et non 64). Cela confirme que le paquet passe par fw01 — chaque routeur décrémente le TTL de 1. C'est la preuve visible que le routage inter-VLAN fonctionne correctement.

Commandes de test

```
# Depuis admin-mgmt01
ping -c 3 192.168.10.254          # Gateway MGMT
ping -c 3 192.168.40.10           # web-dmz01 (DMZ)
curl http://192.168.40.10         # Service web DMZ

# Depuis client01
ping -c 3 192.168.30.254          # Gateway USERS
ping -c 3 192.168.40.10           # web-dmz01 (autorise)
ping -c 3 192.168.20.10           # db-srv01 (bloque)
```

Bilan final

Élément	Description	Statut
Proxmox VE 9.2.3	Interface web accessible sur https://192.168.131.135:8006	OK
ISO Debian 13.5.0	Disponible dans local → Images ISO	OK
Template tmpl-debian-13-base	VM 9000 convertie en template avec cloud-init	OK
QEMU Guest Agent	Installé, IP visible dans Proxmox	OK
web01 (101)	Apache — curl http://192.168.131.138	OK
db01 (102)	MariaDB 11.8.6 actif	OK
admin01 (103)	Tests réseau validés	OK
test-rollback01 (104)	Snapshot + panne + rollback validés	OK
vmbr1 VLAN-aware	Bridge configuré avec 4 VLANs (10/20/30/40)	OK
fw01 (200)	5 interfaces, routage IP, NAT, nftables	OK
web-dmz01 (201)	Apache en zone DMZ (192.168.40.10)	OK
db-srv01 (202)	MariaDB en zone SRV (192.168.20.10)	OK
client01 (203)	Zone USERS (192.168.30.10)	OK
admin-mgmt01 (204)	Zone MGMT (192.168.10.10)	OK
Routage inter-VLAN	MGMT et USERS joignent la DMZ via fw01	OK
Filtrage USERS → SRV	Trafic bloque par nftables sur fw01	OK

Architecture finale

L'environnement Proxmox simule une infrastructure d'entreprise segmentée en 4 zones réseau (MGMT, SRV, USERS, DMZ) interconnectées par un firewall Debian (fw01) appliquant des regles de filtrage nftables. Le routage inter-VLAN est assure par fw01 avec NAT vers Internet. Les clients en zone USERS peuvent acceder au serveur web en DMZ mais sont bloques vers la zone SRV (base de donnees) — principe de defense en profondeur.

ANNEXES (Capture d’ecran marquante)

Capture d’écran de la configuration du firewall avec les différentes cartes réseaux + tag associés (tag = vlan pour la segmentation réseau)

